TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

CHUYÊN ĐỀ TỐT NGHIỆP 1 · HỌC KỲ 1, NĂM HỌC 2026 – 2027

Tài liệu tự học buổi 9 — Trình bày kết quả cho người dùng theo chuyên ngành

Giao diện React/Vue gọi API thật [SE] · Nạp kho dữ liệu và dựng dashboard [DA] · Huấn luyện, so sánh và đánh giá mô hình [AI]. Mỗi phần có mã mẫu hoàn chỉnh trên bộ dữ liệu Mekong Mobile và checklist tự kiểm.

Sản phẩm của tài liệu này là thành phần 3 của Bài tập 2 — module mở rộng và phần trình bày kết quả. Làm ở workshop buổi 10, nộp cuối buổi 12.

Biên soạn: **TS. Nguyễn Trí Hải** · Bộ môn Kỹ thuật phần mềm

Tài liệu lưu hành nội bộ · Bộ tài liệu tự học phần chuyên ngành (30% thời lượng lý thuyết buổi 9). Tài liệu song ngữ: bản tiếng Việt ở nửa đầu, bản tiếng Anh (English version) ở nửa sau.

Cách dùng tài liệu này

Giảng viên giảng trên lớp (buổi 9)	Sinh viên tự đọc trong tài liệu này
Ba thứ phải viết ra ở mỗi ranh giới giữa các thành phần; hợp đồng giao tiếp và vì sao nó cần thiết; mã trạng thái và định dạng lỗi của API; bốn nguyên tắc trình bày kết quả; ba trạng thái giao diện hay bị quên (đang tải, rỗng, lỗi); ba loại comment đáng viết; tích hợp tăng dần.	Cách hiện thực cụ thể phần trình bày kết quả trên stack của track mình: cấu trúc thành phần, mã mẫu lớp gọi API, mã mẫu nạp kho dữ liệu, mã mẫu so sánh mô hình.

Ba điều đúng cho cả ba track — đọc trước khi vào phần của mình

1. BA TRẠNG THÁI, không phải một. Mọi thứ hiển thị kết quả đều phải xử lý ba trạng thái: đang tải · không có dữ liệu · có lỗi. Chỉ làm trạng thái "có dữ liệu" là lỗi bị trừ điểm ở mọi track — giao diện trắng, dashboard trống, hay biểu đồ vẽ trên tập rỗng đều là cùng một lỗi.

2. KHÔNG CÓ DỮ LIỆU GIẢ TRONG BÀI NỘP. Phần trình bày phải lấy dữ liệu từ nguồn thật của hệ thống: API thật, bảng đích thật, mô hình thật. Số liệu ghi cứng trong code là lỗi nghiêm trọng vì nó phá nguyên tắc TRUY VẾT ĐƯỢC.

3. MỌI CON SỐ PHẢI CÓ NGỮ CẢNH SO SÁNH. "87 phiếu" không nói được gì. "87 phiếu, tăng 12% so với tháng trước" hoặc "87 phiếu trên mục tiêu 100" thì nói được. Nguyên tắc này áp dụng cho giao diện [SE], cho dashboard [DA] và cho báo cáo chỉ số mô hình [AI].

PHẦN A — Track SE: Giao diện React/Vue gọi API thật

A1. Cấu trúc thành phần — tách theo màn hình đã vẽ ở wireframe

Cấu trúc dưới đây là cấu trúc tối thiểu. Điểm quan trọng không phải tên thư mục mà là ba sự tách biệt: lớp gọi API tách khỏi thành phần hiển thị · thành phần theo màn hình tách khỏi thành phần dùng lại được · trạng thái tách khỏi cách vẽ.

Cấu trúc thư mục frontend — ví dụ React (Vue tương tự)

```
frontend/
├── src/
│   ├── api/                    # LỚP GỌI API - toàn bộ fetch nằm ở đây
│   │   ├── client.js           # cấu hình chung: base URL, token, xử lý lỗi
│   │   └── tickets.api.js      # các hàm gọi API của nghiệp vụ phiếu
│   ├── hooks/                  # trạng thái và logic lấy dữ liệu
│   │   └── useTickets.js
│   ├── pages/                  # MỘT tệp cho MỘT màn hình trong wireframe
│   │   ├── TicketListPage.jsx
│   │   ├── TicketDetailPage.jsx
│   │   └── ReceiveDevicePage.jsx
│   ├── components/             # thành phần DÙNG LẠI được ở nhiều màn hình
│   │   ├── DataState.jsx       # xử lý 3 trạng thái: loading / empty / error
│   │   ├── Badge.jsx
│   │   └── Pagination.jsx
│   ├── App.jsx
│   └── main.jsx
├── .env.example                # VITE_API_BASE_URL=http://localhost:8000
├── package.json
├── package-lock.json           # BẮT BUỘC commit
└── README.md
```

Quy tắc: không gọi fetch rải rác trong component

Mọi lệnh gọi tới máy chủ đều đi qua src/api/. Component không bao giờ chứa chữ "fetch" hay "axios".

Vì sao: khi đổi base URL, khi thêm token vào header, khi thêm xử lý lỗi 401 — chỉ phải sửa một chỗ chứ không phải mười lăm chỗ.

Phép thử 30 giây: chạy grep -rn "fetch(\|axios" src/pages src/components — kết quả phải trống. Nếu có, hãy chuyển lệnh gọi đó vào src/api/ trước khi tới buổi 10.

A2. Lớp gọi API — mã mẫu hoàn chỉnh

src/api/client.js — một chỗ duy nhất biết về HTTP

```
const BASE = import.meta.env.VITE_API_BASE_URL;   // KHÔNG ghi cứng URL trong code

export class ApiError extends Error {
  constructor(status, code, message) {
    super(message);
    this.status = status;    // mã HTTP
    this.code = code;        // mã nghiệp vụ do backend trả về
  }
}

export async function request(path, options = {}) {
  const token = localStorage.getItem('token');
  const res = await fetch(`${BASE}${path}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...options.headers,
    },
  });
```

```
  if (res.status === 204) return null;

  let body = null;
  try { body = await res.json(); } catch { /* phản hồi không phải JSON */ }

  if (!res.ok) {
    // Dịch lỗi của máy chủ thành lỗi của ứng dụng, KÈM THÔNG ĐIỆP CHO NGƯỜI DÙNG
    throw new ApiError(res.status, body?.error, body?.message
      || 'Không kết nối được tới hệ thống. Vui lòng thử lại.');
  }
  return body?.data ?? body;
}
```

src/api/tickets.api.js — mỗi nghiệp vụ một hàm, tên hàm nói rõ nó làm gì

```
import { request } from './client';

export const ticketsApi = {
  danhSach: ({ trang = 1, trangThai, tuKhoa } = {}) => {
    const q = new URLSearchParams({ trang: String(trang) });
    if (trangThai) q.set('trang_thai', trangThai);
    if (tuKhoa)    q.set('tu_khoa', tuKhoa);
    return request(`/api/tickets?${q}`);
  },
  chiTiet:    id       => request(`/api/tickets/${id}`),
  tiepNhan:   payload  => request('/api/tickets',
                          { method: 'POST', body: JSON.stringify(payload) }),
  phanCong:   (id, ktv) => request(`/api/tickets/${id}/technician`,
                          { method: 'PATCH', body: JSON.stringify({ technicianId: ktv }) }),
};
```

A3. Ba trạng thái — thành phần dùng lại được

Đây là thành phần đáng viết nhất trong toàn bộ frontend, vì nó xoá bỏ ba lỗi phổ biến nhất bằng một lần viết duy nhất.

src/hooks/useTickets.js và src/components/DataState.jsx

```
// ---------------- hooks/useTickets.js ----------------
import { useEffect, useState } from 'react';
import { ticketsApi } from '../api/tickets.api';

export function useTickets(filters) {
  const [state, setState] = useState({ data: null, loading: true, error: null });

  useEffect(() => {
    let huy = false;
    setState({ data: null, loading: true, error: null });
    ticketsApi.danhSach(filters)
      .then(data => { if (!huy) setState({ data, loading: false, error: null }); })
      .catch(err => { if (!huy) setState({ data: null, loading: false, error: err }); });
    return () => { huy = true; };          // tránh cập nhật sau khi component đã tháo
  }, [JSON.stringify(filters)]);

  return state;
}

// ---------------- components/DataState.jsx ----------------
export function DataState({ loading, error, data, empty, children }) {
  if (loading) return <div className="skeleton">Đang tải dữ liệu…</div>;

  if (error) return (
    <div className="alert alert-error">
      <p>{error.message}</p>
      {/* NẾU lỗi là 422 hoặc 409 thì là lỗi nghiệp vụ - nói rõ người dùng phải làm gì */}
      <button onClick={() => window.location.reload()}>Thử lại</button>
    </div>
  );

  const rong = !data || (Array.isArray(data) && data.length === 0);
```

```
  if (rong) return (
    <div className="empty-state">
      <p>{empty?.message || 'Chưa có dữ liệu nào.'}</p>
      {empty?.action}
    </div>
  );

  return children(data);
}

// ---------------- pages/TicketListPage.jsx (cách dùng) ----------------
export default function TicketListPage() {
  const [filters, setFilters] = useState({ trang: 1 });
  const { data, loading, error } = useTickets(filters);

  return (
    <section>
      <h1>Danh sách phiếu bảo hành</h1>
      <FilterBar value={filters} onChange={setFilters} />
      <DataState loading={loading} error={error} data={data}
        empty={{ message: 'Không có phiếu nào khớp bộ lọc hiện tại.',
                 action: <button onClick={() => setFilters({ trang: 1 })}>Xoá bộ lọc</button> }}>
        {rows => <TicketTable rows={rows} />}
      </DataState>
    </section>
  );
}
```

Vì sao trạng thái RỖNG phải phân biệt được hai trường hợp

Rỗng vì chưa có dữ liệu nào: "Chưa có phiếu bảo hành nào. Bấm Tiếp nhận thiết bị để tạo phiếu đầu tiên." — người dùng cần một hành động để bắt đầu.

Rỗng vì bộ lọc không ra kết quả: "Không có phiếu nào khớp bộ lọc hiện tại." kèm nút xoá bộ lọc — người dùng cần biết vấn đề nằm ở bộ lọc, không phải ở hệ thống.

Hai trường hợp này nhìn giống nhau trên màn hình nếu chỉ viết "Không có dữ liệu", và người dùng sẽ nghĩ hệ thống hỏng.

Câu hỏi Giảng viên sẽ hỏi ở buổi 10: "Hãy lọc theo một trạng thái không có phiếu nào." Sinh viên cần chuẩn bị để màn hình hiện đúng thông báo, không phải bảng trắng.

A4. Lưu và dùng token an toàn ở mức prototype

Cách lưu token	Rủi ro	Có được dùng trong Bài tập 2?
Ghi cứng trong mã nguồn	Token nằm trong repo, ai đọc repo cũng dùng được. Đây là lỗi bảo mật nghiêm trọng nhất và bị trừ điểm ngay.	KHÔNG. Tuyệt đối không.
localStorage	Bị đọc được nếu trang có lỗ hổng chèn mã (XSS). Chấp nhận được ở mức prototype nếu ứng dụng không hiển thị nội dung do người dùng nhập mà chưa xử lý.	Được, kèm ghi nhận hạn chế này vào tài liệu.
Cookie HttpOnly do máy chủ đặt	An toàn hơn vì mã JavaScript không đọc được. Cần cấu hình phía máy chủ và xử lý CORS.	Được và được khuyến khích. Cộng điểm ở tiêu chí chất lượng kỹ thuật.

Ba việc bắt buộc về xác thực trong Bài tập 2

Base URL của API đặt trong biến môi trường, không ghi cứng. Có .env.example ghi rõ biến này.

Cách lưu token	Rủi ro	Có được dùng trong Bài tập 2?

Khi nhận phản hồi 401, tự động xoá token và chuyển về trang đăng nhập — không để người dùng nhìn màn hình lỗi mãi.

Ghi vào tài liệu một dòng nêu rõ cách lưu token đã chọn và hạn chế của cách đó. Nhận biết được hạn chế của lựa chọn của mình là năng lực CLO5.

A5. Checklist tự kiểm trước khi tới workshop buổi 10 [SE]

- ☐ Có ít nhất ba màn hình chính hoạt động, lấy dữ liệu từ API thật (không phải dữ liệu ghi cứng).
- ☐ grep tìm "fetch(" và "axios" trong pages và components trả về kết quả trống.
- ☐ Cả ba trạng thái đều thấy được: tắt máy chủ backend rồi mở trang → hiện thông báo lỗi có nút thử lại.
- ☐ Lọc theo điều kiện không có kết quả → hiện thông báo rỗng riêng kèm nút xoá bộ lọc.
- ☐ Base URL API lấy từ biến môi trường; có .env.example.
- ☐ Nhận 401 thì xoá token và chuyển về trang đăng nhập.
- ☐ Đã thử trên khung màn hình rộng và khung màn hình hẹp (khoảng 375px) — bảng không bị tràn ngang.
- ☐ Đã commit package-lock.json.
- ☐ README ghi lệnh cài đặt và lệnh chạy frontend, kèm ghi chú backend phải chạy trước.

PHẦN B — Track DA: Nạp kho dữ liệu và dựng dashboard

B1. Từ staging tới bảng đích — nạp lại được bằng khoá nghiệp vụ

Ở buổi 7, Sinh viên đã đưa dữ liệu đã làm sạch vào bảng staging. Bước tiếp theo là nạp vào bảng đích theo lược đồ kho dữ liệu đã thiết kế ở Bài tập 1. Yêu cầu quan trọng nhất: nạp lại lần thứ hai không được nhân đôi dữ liệu.

sql/010_load_dim_customer.sql — nạp dimension bằng khoá nghiệp vụ

```
-- Khoá nghiệp vụ (business key) của khách hàng: số điện thoại đã chuẩn hoá.
-- Khoá thay thế (surrogate key) customer_sk do kho dữ liệu tự sinh.
INSERT INTO dim_customer (business_key, ho_ten, sdt_chuan, tinh_thanh, ngay_tao, cap_nhat_luc)
SELECT s.khoa_hop_nhat, s.ho_ten, s.sdt_chuan, s.tinh_thanh, s.ngay_tao, NOW()
FROM   staging_customers s
ON CONFLICT (business_key) DO UPDATE
SET    ho_ten      = EXCLUDED.ho_ten,
       sdt_chuan   = EXCLUDED.sdt_chuan,
       tinh_thanh  = EXCLUDED.tinh_thanh,
       cap_nhat_luc = NOW()
-- Chỉ cập nhật khi có thay đổi thật, tránh ghi lại vô ích mỗi lần chạy
WHERE  dim_customer.ho_ten     IS DISTINCT FROM EXCLUDED.ho_ten
   OR  dim_customer.sdt_chuan  IS DISTINCT FROM EXCLUDED.sdt_chuan
   OR  dim_customer.tinh_thanh IS DISTINCT FROM EXCLUDED.tinh_thanh;
```

sql/020_load_fact_ticket.sql — nạp fact theo lô, xoá rồi nạp lại

```
-- Fact nạp theo LÔ NGÀY. Trước khi nạp, xoá mọi dòng của lô đó.
-- Cách này bảo đảm chạy lại được và đơn giản hơn nhiều so với so sánh từng dòng.
BEGIN;

DELETE FROM fact_ticket
WHERE  ngay_tiep_nhan_key BETWEEN :tu_ngay AND :den_ngay;

INSERT INTO fact_ticket (
  ngay_tiep_nhan_key, customer_sk, product_sk, center_sk, category_sk,
  so_phieu, so_gio_xu_ly, tre_han, chi_phi_linh_kien)
SELECT
  TO_CHAR(t.received_at, 'YYYYMMDD')::INT,
  dc.customer_sk, dp.product_sk, dsc.center_sk, dic.category_sk,
  1                                                  AS so_phieu,
  EXTRACT(EPOCH FROM (t.closed_at - t.received_at))/3600 AS so_gio_xu_ly,
  CASE WHEN t.closed_at > t.due_date THEN 1 ELSE 0 END   AS tre_han,
  COALESCE(lk.chi_phi, 0)                                AS chi_phi_linh_kien
FROM staging_tickets t
JOIN dim_customer      dc  ON dc.business_key  = t.khoa_khach_hang
JOIN dim_product       dp  ON dp.business_key  = t.product_code
JOIN dim_service_center dsc ON dsc.business_key = t.center_code
JOIN dim_issue_category dic ON dic.business_key = t.category_name
LEFT JOIN v_chi_phi_linh_kien lk ON lk.ticket_code = t.ticket_code
WHERE t.received_at::DATE BETWEEN :tu_ngay AND :den_ngay;

COMMIT;
```

Bốn điều phải đúng ở bảng fact — kiểm trước khi vẽ biểu đồ

1. GRAIN rõ ràng và ghi vào tài liệu. "Một dòng fact = một phiếu bảo hành" là grain. Nếu không nói được grain bằng một câu, lược đồ chưa đúng và mọi biểu đồ phía sau đều đáng ngờ.

2. Measure phải cộng dồn được. so_phieu và chi_phi_linh_kien cộng được. Nhưng "tỉ lệ trễ hạn" thì KHÔNG cộng được — nên lưu tre_han là 0/1 rồi tính tỉ lệ bằng AVG lúc truy vấn, đừng lưu sẵn tỉ lệ.

3. Dùng khoá thay thế, không dùng khoá nghiệp vụ trong fact. Fact trỏ tới dim bằng customer_sk. Nhờ vậy khi số điện thoại khách hàng đổi, fact không bị ảnh hưởng.

4. JOIN không được làm mất dòng. Đếm số dòng staging và số dòng fact sau khi nạp. Nếu fact ít hơn, có dòng bị JOIN loại bỏ — phải tìm ra và xử lý bằng dòng "không xác định" trong dim, không được âm thầm bỏ.

B2. Dashboard — hai biểu đồ trả lời hai câu hỏi cụ thể

Bài tập 2 yêu cầu ít nhất hai biểu đồ. Quan trọng không phải số lượng mà là: mỗi biểu đồ phải trả lời một câu hỏi phân tích viết ra được thành câu, và phải có ngữ cảnh so sánh.

Câu hỏi phân tích	Loại biểu đồ phù hợp	Ngữ cảnh so sánh phải có	Vì sao chọn loại này
Trung tâm nào đang xử lý chậm hơn các trung tâm khác?	Cột ngang, sắp xếp theo giá trị giảm dần	Đường trung bình toàn hệ thống, hoặc mục tiêu SLA	So sánh giữa các hạng mục rời rạc thì cột ngang dễ đọc nhãn hơn cột dọc.
Tỉ lệ trễ hạn thay đổi thế nào qua sáu tháng?	Đường theo thời gian	Cùng kỳ năm trước, hoặc ngưỡng mục tiêu vẽ thành đường ngang	Xu hướng theo thời gian thì đường thể hiện được chiều biến động, cột thì không.
Nhóm lỗi nào chiếm nhiều chi phí linh kiện nhất?	Cột ngang, kèm phần trăm tích luỹ	Tổng chi phí, để đọc được tỉ lệ đóng góp	Cho thấy ngay vài nhóm lỗi nào chiếm phần lớn chi phí.
Kỹ thuật viên nào đang quá tải?	Bảng có thanh trong ô, hoặc cột ngang có ngưỡng	Ngưỡng 8 phiếu mở theo quy tắc nghiệp vụ QT-11	Ngưỡng nghiệp vụ vẽ thành đường thì người xem biết ngay ai vượt.

src/dashboard/app.py — Streamlit, ba trạng thái và ngữ cảnh so sánh

```
import streamlit as st
import pandas as pd
import plotly.express as px
from src.db import doc_sql

st.set_page_config(page_title='Mekong Mobile – Bảo hành', layout='wide')
st.title('Hiệu quả xử lý bảo hành')

# --- Bộ lọc ---
c1, c2 = st.columns(2)
tu_ngay  = c1.date_input('Từ ngày',  pd.Timestamp('2026-01-01'))
den_ngay = c2.date_input('Đến ngày', pd.Timestamp('2026-06-30'))

@st.cache_data(ttl=300)          # cache 5 phút - GHI RÕ trong tài liệu
def lay_du_lieu(tu, den):
    return doc_sql('sql/q_hieu_qua_trung_tam.sql', {'tu_ngay': tu, 'den_ngay': den})

# --- TRẠNG THÁI 1: đang tải ---
with st.spinner('Đang truy vấn kho dữ liệu…'):
    try:
        df = lay_du_lieu(tu_ngay, den_ngay)
    # --- TRẠNG THÁI 3: lỗi ---
    except Exception as e:
        st.error('Không đọc được dữ liệu từ kho. Kiểm tra kết nối và thử lại.')
        st.caption(f'Chi tiết kỹ thuật: {type(e).__name__}')   # KHÔNG in chuỗi kết nối
        st.stop()

# --- TRẠNG THÁI 2: rỗng ---
if df.empty:
    st.info('Không có phiếu nào trong khoảng thời gian đã chọn. '
            'Thử mở rộng khoảng ngày hoặc kiểm tra pipeline đã nạp dữ liệu chưa.')
    st.stop()

# --- Có dữ liệu: vẽ biểu đồ KÈM NGỮ CẢNH SO SÁNH ---
```

Câu hỏi phân tích	Loại biểu đồ phù hợp	Ngữ cảnh so sánh phải có	Vì sao chọn loại này

```
tb_he_thong = df['gio_xu_ly_tb'].mean()

fig = px.bar(df.sort_values('gio_xu_ly_tb'),
             x='gio_xu_ly_tb', y='ten_trung_tam', orientation='h',
             labels={'gio_xu_ly_tb': 'Giờ xử lý trung bình',
                     'ten_trung_tam': ''})
# Đường tham chiếu: đây là phần biến một biểu đồ thành một kết luận
fig.add_vline(x=tb_he_thong, line_dash='dash',
              annotation_text=f'Trung bình hệ thống {tb_he_thong:.1f} giờ')
st.plotly_chart(fig, use_container_width=True)
st.caption(f'Nguồn: fact_ticket · {len(df)} trung tâm · '
           f'số liệu tính tới {df["cap_nhat_luc"].max():%d/%m/%Y %H:%M}')
```

Ba điều mọi biểu đồ trong Bài tập 2 phải có

1. Nhãn trục có đơn vị. "Giờ xử lý trung bình" chứ không phải "gio_xu_ly_tb". Tên cột trong cơ sở dữ liệu không phải nhãn cho người xem.

2. Một đường hoặc mốc tham chiếu. Trung bình, mục tiêu, kỳ trước, hoặc ngưỡng nghiệp vụ. Không có mốc thì người xem không kết luận được gì.

3. Dòng ghi nguồn và thời điểm dữ liệu. "Nguồn: fact_ticket · số liệu tính tới 30/06/2026 14:20". Đây là nguyên tắc TRUY VẾT ĐƯỢC áp dụng vào dashboard, và là điều phân biệt một dashboard dùng được với một hình vẽ đẹp.

B3. Chạy pipeline theo lịch — ở mức phạm vi học phần

Học phần này không yêu cầu hệ thống điều phối đầy đủ (Airflow, Dagster) — đó là nội dung của Chuyên đề tốt nghiệp 2. Ở mức Bài tập 2, chạy theo lịch nghĩa là: pipeline chạy được bằng một lệnh có tham số ngày, và cách hẹn lịch cho lệnh đó được ghi vào tài liệu.

Mức phạm vi được chấp nhận cho Bài tập 2

```
# 1. Pipeline nhận tham số ngày, mặc định là hôm qua
python -m src.pipeline --tu-ngay 2026-06-01 --den-ngay 2026-06-30
python -m src.pipeline                # mặc định: hôm qua

# 2. Ghi log mỗi lần chạy vào một tệp, KÈM mã lô
#    reports/run_20260701_0200.log

# 3. Ghi vào Deployment Guide cách hẹn lịch (không bắt buộc cài đặt thật)
#    Linux/macOS - crontab:
#      0 2 * * *  cd /opt/du-an && ./venv/bin/python -m src.pipeline >> logs/cron.log 2>&1
#    Windows - Task Scheduler: tạo task chạy hằng ngày 02:00 với lệnh tương đương

# 4. Xử lý khi lần chạy trước thất bại: pipeline PHẢI chạy lại được,
#    nên chạy bù bằng cách gọi lại với đúng khoảng ngày đó.
python -m src.pipeline --tu-ngay 2026-06-30 --den-ngay 2026-06-30   # chạy bù
```

B4. Checklist tự kiểm trước khi tới workshop buổi 10 [DA]

- ☐ Bảng đích đã có dữ liệu; đếm số dòng và so với số dòng staging, sai lệch giải thích được.
- ☐ Nạp lần thứ hai không nhân đôi dữ liệu (đã chạy thử và đếm lại).
- ☐ Grain của bảng fact viết ra được bằng một câu và đã ghi vào Data dictionary.
- ☐ Mọi measure trong fact đều cộng dồn được; không lưu sẵn tỉ lệ hay phần trăm.
- ☐ Có ít nhất hai biểu đồ, mỗi biểu đồ ứng với một câu hỏi phân tích viết ra được.
- ☐ Mỗi biểu đồ có nhãn trục kèm đơn vị, một mốc tham chiếu, và dòng ghi nguồn kèm thời điểm.

- ☐ Dashboard xử lý đúng cả ba trạng thái: đang tải, rỗng (bộ lọc không ra dòng nào), lỗi kết nối.
- ☐ Thông báo lỗi trên dashboard KHÔNG in ra chuỗi kết nối hay tên bảng bên trong.
- ☐ Pipeline nhận tham số ngày và ghi log mỗi lần chạy vào reports/.
- ☐ Deployment Guide có mục nêu cách hẹn lịch chạy pipeline.

PHẦN C — Track AI: Huấn luyện, so sánh và đánh giá mô hình

C1. Quy tắc quan trọng nhất: tập test chỉ dùng MỘT lần

Ba tập, ba vai trò khác nhau — không được lẫn

Tập train (70%): dùng để mô hình học tham số. Được dùng bao nhiêu lần cũng được.

Tập validation (15%): dùng để CHỌN — chọn mô hình nào, chọn siêu tham số nào, chọn cách làm sạch nào. Được dùng nhiều lần.

Tập test (15%): dùng ĐÚNG MỘT LẦN, ở cuối cùng, để báo cáo kết quả. Mỗi lần xem kết quả trên tập test rồi quay lại sửa mô hình, tập test mất giá trị vì nó đã ảnh hưởng tới việc lựa chọn mô hình.

Vì sao đây là vấn đề trung thực chứ không chỉ là quy ước: nếu thử mười mô hình trên tập test rồi báo cáo mô hình tốt nhất, con số được báo cáo là kết quả tốt nhất trong mười lần thử, không phải năng lực thật của mô hình trên dữ liệu mới. Báo cáo nó như năng lực thật là sai lệch số liệu.

C2. So sánh ít nhất hai mô hình — trên tập validation

src/models/train.py — so sánh trên validation, đo một lần trên test

```
import json, joblib
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import ComplementNB
from sklearn.dummy import DummyClassifier
from sklearn.metrics import f1_score, classification_report, confusion_matrix

def huan_luyen_va_chon(X_tr, y_tr, X_val, y_val, hat_giong=42):
    ung_vien = {
        'dummy_da_so':  DummyClassifier(strategy='most_frequent'),
        'logistic':     LogisticRegression(max_iter=1000, class_weight='balanced',
                                           random_state=hat_giong),
        'linear_svc':   LinearSVC(class_weight='balanced', random_state=hat_giong),
        'complement_nb': ComplementNB(),
    }
    ket_qua = {}
    for ten, mo_hinh in ung_vien.items():
        mo_hinh.fit(X_tr, y_tr)
        # CHỌN trên VALIDATION - không bao giờ trên test
        f1 = f1_score(y_val, mo_hinh.predict(X_val), average='macro', zero_division=0)
        ket_qua[ten] = {'f1_macro_val': round(float(f1), 4)}
        print(f'{ten:16s} F1 macro (val) = {f1:.4f}')

    ten_tot_nhat = max(
        (t for t in ket_qua if t != 'dummy_da_so'),
        key=lambda t: ket_qua[t]['f1_macro_val'])
    return ung_vien[ten_tot_nhat], ten_tot_nhat, ket_qua

def do_lan_cuoi(mo_hinh, X_te, y_te, nhan_labels):
    """Gọi ĐÚNG MỘT LẦN, sau khi đã chọn xong mô hình."""
    y_pred = mo_hinh.predict(X_te)
    return {
        'f1_macro_test': round(float(f1_score(y_te, y_pred, average='macro',
                                              zero_division=0)), 4),
        'f1_theo_nhan': {
            nhan: round(float(f), 4) for nhan, f in zip(
                nhan_labels,
                f1_score(y_te, y_pred, average=None, labels=nhan_labels,
                         zero_division=0))},
        'confusion_matrix': confusion_matrix(y_te, y_pred,
                                             labels=nhan_labels).tolist(),
        'nhan_labels': list(nhan_labels),
        'bao_cao_chi_tiet': classification_report(y_te, y_pred, zero_division=0,
                                                  output_dict=True),
    }
```

C3. Chọn chỉ số đánh giá — và giải thích được lý do chọn

Chỉ số	Đo cái gì	Dùng khi nào	Cạm bẫy
Accuracy (độ chính xác)	Tỉ lệ dự đoán đúng trên tổng số	Các nhãn cân bằng và mọi loại sai đều tệ như nhau	Với dữ liệu lệch 90/10, đoán bừa nhãn đa số đã cho 90% — con số cao nhưng mô hình vô dụng.
F1 macro	Trung bình đều F1 của mọi nhãn	Nhãn mất cân bằng và mọi nhãn đều quan trọng như nhau	Nhãn ít mẫu ảnh hưởng mạnh tới chỉ số, nên số liệu dao động nhiều giữa các lần chia dữ liệu.
F1 weighted	Trung bình F1 có trọng số theo số mẫu	Muốn một con số phản ánh trải nghiệm chung của người dùng	Che giấu việc mô hình bỏ rơi hoàn toàn nhãn ít mẫu — gần với accuracy hơn là F1 macro.
Precision cho một nhãn	Trong các mẫu được dự đoán là nhãn đó, bao nhiêu phần đúng	Khi báo động sai gây tốn kém (gọi kỹ thuật viên đi sai địa chỉ)	Có thể rất cao nếu mô hình chỉ dự đoán nhãn đó khi cực kỳ chắc, bỏ sót phần lớn ca thật.
Recall cho một nhãn	Trong các mẫu thật là nhãn đó, bao nhiêu phần được tìm ra	Khi bỏ sót gây tốn kém (bỏ sót thiết bị vào nước dẫn tới hỏng nặng)	Có thể rất cao nếu mô hình gán nhãn đó cho gần như mọi mẫu.
RMSE / MAE	Sai số trung bình với bài toán hồi quy	Dự đoán số liên tục (thời gian xử lý, chi phí)	RMSE bị các trường hợp ngoại lai chi phối mạnh; hãy báo cáo cả MAE bên cạnh.

Câu trả lời mẫu cho câu hỏi "vì sao chọn chỉ số này"

Câu trả lời yếu: "Dùng F1 macro vì nó phổ biến với bài toán phân loại."

Câu trả lời tốt: "Dữ liệu bị lệch: nhãn KHÁC chiếm 31% còn nhãn NƯỚC_VÀO chỉ 9%. Nếu dùng accuracy thì mô hình đoán bừa nhãn đa số đã đạt 31%, nên con số đó không nói được gì. F1 macro được chọn vì nó tính trung bình đều cho cả sáu nhãn, nên nếu mô hình bỏ rơi nhãn NƯỚC_VÀO thì chỉ số tụt rõ. Về nghiệp vụ, bỏ sót ca vào nước là tốn kém nhất nên báo cáo còn nêu riêng recall của nhãn đó."

Điểm khác biệt: câu trả lời tốt nối chỉ số về (1) đặc điểm dữ liệu thật, có số liệu; và (2) hệ quả nghiệp vụ. Đó là truy vết được.

Đây chính là dạng câu hỏi của câu 2c đề thi cuối kỳ và của bộ ba câu hỏi chuẩn ở buổi 10.

C4. Đọc confusion matrix — biến bảng số thành kết luận

Confusion matrix là công cụ chẩn đoán mạnh nhất trong tay Sinh viên, nhưng chỉ khi được đọc kỹ chứ không chỉ dán vào báo cáo. Cách đọc: mỗi hàng là nhãn thật, mỗi cột là nhãn dự đoán. Ô ngoài đường chéo là các lần nhầm.

Ví dụ confusion matrix và cách đọc thành ba kết luận

```
Nhãn dự đoán →    MÀN_HÌNH  PIN   SẠC   PHẦN_MỀM  NƯỚC_VÀO  KHÁC
Nhãn thật
  MÀN_HÌNH            142    3     2         4          6       11
  PIN                   4  118    31         3          1       14
  SẠC                   2   28    96         2          0       12
  PHẦN_MỀM              5    4     3        88          1       29
  NƯỚC_VÀO              9    2     1         2         41       16
  KHÁC                 14   12    10        24          7      113

BA KẾT LUẬN ĐỌC ĐƯỢC TỪ BẢNG NÀY:
```

```
1. Cặp nhầm nhiều nhất là PIN ↔ SẠC (31 + 28 = 59 lần).
   Ý nghĩa nghiệp vụ: mô tả sự cố pin và sự cố sạc dùng nhiều từ giống nhau
   ("không lên nguồn", "sạc không vào"). Hướng xử lý: bổ sung đặc trưng
   phân biệt, hoặc gộp hai nhãn lại nếu nghiệp vụ chấp nhận được - và
   NẾU GỘP thì phải giải trình với người dùng nghiệp vụ, không tự quyết.

2. Nhãn NƯỚC_VÀO có recall thấp nhất: 41/(9+2+1+2+41+16) = 57%.
   Ý nghĩa nghiệp vụ: đây là nhãn TỐN KÉM NHẤT khi bỏ sót (thiết bị vào nước
   để lâu sẽ hỏng nặng hơn). 43% ca bị bỏ sót là rủi ro nghiệp vụ thật,
   phải ghi rõ trong báo cáo và trong model card.

3. Nhãn KHÁC hút về nhiều dự đoán sai từ mọi nhãn khác (cột KHÁC: 11+14+12+29+16).
   Ý nghĩa: KHÁC đang làm nhãn thùng rác. Hướng xử lý: xem lại định nghĩa nhãn
   cùng người dùng nghiệp vụ, hoặc loại KHÁC khỏi chỉ số chính và báo cáo riêng.
```

Ba câu phải viết trong báo cáo Bài tập 2 sau khi đọc confusion matrix

Cặp nhãn bị nhầm nhiều nhất là gì, và điều đó có ý nghĩa nghiệp vụ gì?

Nhãn nào có recall thấp nhất, và bỏ sót nhãn đó gây hậu quả gì trên thực tế?

Có nhãn nào đang làm "thùng rác" hút về mọi dự đoán không chắc chắn không?

Ba câu này là phần có giá trị nhất trong báo cáo. Một bảng số dán vào báo cáo mà không có ba câu này thì gần như không được điểm ở tiêu chí chất lượng kỹ thuật.

C5. Lưu mô hình — nhưng không commit tệp mô hình

Lưu và nạp lại mô hình đúng cách

```
import joblib, json, pathlib

# --- Lưu: LƯU CẢ vectorizer VÀ mô hình, cùng một lúc, cùng một phiên bản ---
pathlib.Path('artifacts').mkdir(exist_ok=True)
joblib.dump({'vectorizer': vec, 'model': mo_hinh_tot_nhat,
             'nhan_labels': list(nhan_labels),
             'hat_giong': cfg['hat_giong'],
             'phien_ban_du_lieu': cfg['phien_ban_du_lieu']},
            'artifacts/model_v1.joblib')

# --- .gitignore PHẢI có: ---
# artifacts/*.joblib
# artifacts/*.pkl

# --- Thay vào đó, commit reports/metrics.json (tệp văn bản nhỏ, đọc được) ---
# và ghi trong README lệnh sinh lại mô hình:
#   python -m src.models.train --config configs/train.yaml
```

Vì sao không commit tệp mô hình vào repo

Tệp mô hình thường vài chục megabyte và là tệp nhị phân — repo phình to và mọi thay đổi đều lưu lại một bản đầy đủ mới.

Tệp nhị phân không xem được khác biệt giữa hai phiên bản, nên nó không thuộc về hệ thống quản lý phiên bản mã nguồn.

Thay vào đó: commit script sinh ra mô hình + hạt giống ngẫu nhiên + phiên bản thư viện chính xác. Người khác chạy lại script sẽ có đúng mô hình đó. Đây là nguyên tắc TÁI LẬP ĐƯỢC.

Nếu bắt buộc phải nộp tệp mô hình, nộp qua LMS như một tệp đính kèm riêng, không đưa vào repo.

C6. Checklist tự kiểm trước khi tới workshop buổi 10 [AI]

- ☐ Đã huấn luyện mô hình cơ sở đoán nhãn đa số và ghi con số của nó vào metrics.json.

- ☐ Đã huấn luyện và so sánh ít nhất hai mô hình thật.
- ☐ Việc CHỌN mô hình dựa trên tập validation, không dựa trên tập test.
- ☐ Tập test chỉ được đo một lần, sau khi đã chọn xong mô hình.
- ☐ metrics.json có F1 macro, F1 theo từng nhãn, và confusion matrix.
- ☐ Đã viết ba câu kết luận đọc từ confusion matrix (cặp nhầm nhiều nhất, nhãn recall thấp nhất, nhãn thùng rác).
- ☐ Giải thích được bằng lời vì sao chọn chỉ số đánh giá đó, có nối về số liệu phân bố nhãn thật.
- ☐ Đã ghi hạt giống ngẫu nhiên và đã chạy thử hai lần cho kết quả giống nhau.
- ☐ Lưu cả vectorizer và mô hình trong cùng một tệp artifacts, và .gitignore đã chặn tệp đó.
- ☐ requirements.txt ghi phiên bản chính xác bằng dấu ==.

PHẦN D — Phần chung cho cả ba track: hợp đồng ở ranh giới và tích hợp tăng dần

D1. Viết hợp đồng dữ liệu cho ranh giới quan trọng nhất của module

Buổi 9 trên lớp đã nêu ba thứ mọi hợp đồng phải viết ra: HÌNH DẠNG (trường, kiểu, bắt buộc), Ý NGHĨA (giá trị và đơn vị nghĩa là gì) và HÀNH VI KHI SAI (không có dữ liệu thì trả gì, lỗi báo thế nào). Mẫu dưới đây dùng được cho cả ba track: đặt nó ngay trên hàm chuyển đổi ở ranh giới, và viết một test kiểm tra đúng những gì comment đã hứa.

Mẫu comment hợp đồng — SE (JavaScript), DA và AI (Python)

```
// [SE] src/service/ticket.contract.js
// HỢP ĐỒNG: một phiếu trả ra từ lớp nghiệp vụ cho lớp trình bày.
//   hanCamKet   : chuỗi ISO 8601 kèm múi giờ, luôn có
//   kyThuatVien : chuỗi HOẶC null nếu chưa phân công
//   tenKhach    : chuỗi, đã che số điện thoại theo QT-15
// KHI SAI: không tìm thấy phiếu -> ném NotFoundError (controller trả 404).

# [DA] src/transform/schema.py
# HỢP ĐỒNG: DataFrame phiếu sau bước extract, trước bước transform.
#   ticket_code : str, duy nhất, không rỗng
#   received_at : datetime có múi giờ Asia/Ho_Chi_Minh
#   so_gio_xu_ly: float, đơn vị GIỜ, NaN nếu phiếu chưa đóng
# KHI SAI: thiếu cột -> dừng pipeline, ghi log; dòng lỗi -> reports/reject.csv

# [AI] src/features/build.py
# HỢP ĐỒNG: build_features(df) dùng CHUNG cho huấn luyện và suy luận.
#   Đầu ra  : đúng FEATURE_COLUMNS, đúng thứ tự, không có NaN
# KHI SAI: thiếu cột mo_ta_loi -> KeyError; mô tả rỗng -> chuỗi rỗng, không bỏ dòng
```

D2. Tích hợp tăng dần — ghép từng mối nối

Luồng chính của BT2 đi qua nhiều mối nối. Ghép tất cả rồi mới chạy (big-bang) cho ra một lỗi không biết đến từ đâu. Ghép từng mối nối một, chạy thử sau mỗi lần ghép, thì khi vỡ, lỗi nằm ở mối nối vừa thêm. Phần nào phía sau chưa xong thì tạm thay bằng một stub trả dữ liệu mẫu đúng hợp đồng.

Chuỗi mối nối của luồng chính theo track

[SE] giao diện → client API → controller → service → repository → CSDL. Stub điển hình: một API giả trả đúng hình dạng phiếu để dựng giao diện trước.

[DA] tệp nguồn → extract → staging → transform → bảng đích → dashboard. Stub điển hình: một bảng fact nhỏ nạp tay để dựng dashboard trong lúc pipeline chưa xong.

[AI] dữ liệu → build_features → mô hình → metrics.json (→ API suy luận nếu có). Stub điển hình: DummyClassifier đứng tạm chỗ mô hình thật để nối toàn luồng.

Lưu ý: stub là giàn giáo tạm thời. Bài nộp BT2 KHÔNG được còn stub ở luồng chính.

D3. Checklist chung trước khi tới workshop buổi 10

- ☐ Ranh giới quan trọng nhất của module đã có comment hợp đồng nêu đủ hình dạng, ý nghĩa và hành vi khi sai.
- ☐ Có một hàm chuyển đổi hoặc chuẩn hoá ở ranh giới; đổi tên một trường chỉ phải sửa một chỗ.
- ☐ Dữ liệu đầu vào được kiểm tra ở ranh giới (zod, Joi, pydantic, pandera hoặc assert), không rải if khắp lớp nghiệp vụ.

- ☐ [SE] API trả lỗi theo một định dạng thống nhất gồm error, message và fields; lỗi người dùng trả 4xx, không trả 500.
- ☐ Có ít nhất một test hợp đồng: kiểm tra tên trường, định dạng ngày, giá trị null [SE]; tên cột [DA]; thứ tự cột [AI].
- ☐ Luồng chính đã được ghép tăng dần ít nhất hai mối nối liền kề và chạy thử thành công.

So sánh nhanh ba track — bảng đối chiếu để tự kiểm

Tiêu chí	Track SE	Track DA	Track AI
Sản phẩm của buổi 9–10	≥ 3 màn hình gọi API thật	Bảng đích đã nạp + ≥ 2 biểu đồ	So sánh ≥ 2 mô hình + báo cáo chỉ số
Ba trạng thái thể hiện ở đâu	Component DataState: loading / empty / error	Dashboard: spinner / thông báo rỗng / thông báo lỗi	Không áp dụng cho giao diện, nhưng áp dụng cho báo cáo: thiếu dữ liệu nhãn nào thì phải nói rõ
Mục kiểm chứng "nạp lại được"	Không áp dụng	Nạp lần hai không nhân đôi dòng	Hai lần chạy cho metrics.json giống nhau
Ngữ cảnh so sánh là gì	Số liệu trên màn hình có kỳ trước hoặc mục tiêu	Mốc tham chiếu trên biểu đồ	Mô hình cơ sở để so sánh
Lỗi nghiêm trọng nhất	Dữ liệu ghi cứng trong component thay vì gọi API	Biểu đồ không có mốc tham chiếu và không ghi nguồn	Dùng tập test để chọn mô hình
Câu hỏi Giảng viên hay hỏi ở buổi 10	"Tắt backend đi rồi mở trang xem sao?"	"Số liệu này tính tới thời điểm nào?"	"Vì sao chọn chỉ số này mà không chọn accuracy?"

Ba việc làm ngay trong tuần này, trước workshop buổi 10

1. Làm cho được TRẠNG THÁI LỖI của track mình. [SE] tắt backend rồi mở trang · [DA] đổi sai chuỗi kết nối rồi mở dashboard · [AI] chạy train với tệp dữ liệu không tồn tại. Cả ba đều phải cho ra thông báo rõ ràng, không phải một vệt lỗi dài.

2. Viết ra thành câu MỘT câu hỏi mà phần trình bày của mình trả lời. Nếu không viết được thành câu, phần trình bày đó chưa có mục đích và cần xem lại.

3. Thêm MỘT mốc tham chiếu. Một đường trung bình, một mục tiêu, một mô hình cơ sở. Chỉ mất mười lăm phút và nó là thứ biến số liệu thành một kết luận.

VAN LANG UNIVERSITY · FACULTY OF INFORMATION TECHNOLOGY
SPECIALIZED GRADUATION TOPIC 1 · SEMESTER 1, ACADEMIC YEAR 2026 – 2027

Self-study guide, Session 9 — Presenting results to users by specialization

React/Vue front end calling a real API [SE] · Loading the data warehouse and building a dashboard [DA] · Training, comparing and evaluating models [AI]. Each part includes complete sample code on the Mekong Mobile dataset and a self-check list.

The output of this guide is component 3 of Assignment 2 — the extended module and the presentation of results. Work on it in the Session 10 workshop; submit at the end of Session 12.

Prepared by: **Dr. Nguyễn Trí Hải** · Department of Software Engineering

For internal circulation · Self-study series for the specialization part (30% of Session 9 theory time). Bilingual edition: this is the English version. In the sample code, identifiers and user-facing strings stay in Vietnamese because the case-study system serves Vietnamese users; code comments are translated.

How to use this guide

Taught by the lecturer in class (Session 9)	Read by students in this guide
The three things to write down at every boundary between components; the communication contract and why it is needed; HTTP status codes and the API error format; the four principles for presenting results; the three often-forgotten UI states (loading, empty, error); the three kinds of comments worth writing; incremental integration.	How to implement the presentation of results on your own track's stack: component structure, sample API-client code, sample warehouse-loading code, sample model-comparison code.

Three things that hold for all three tracks — read before your own part

1. THREE STATES, not one. Everything that displays results must handle three states: loading · no data · error. Implementing only the "has data" state is a deduction in every track — a blank screen, an empty dashboard and a chart drawn on an empty set are all the same mistake.

2. NO FAKE DATA IN THE SUBMISSION. The presentation must take data from the system's real sources: the real API, the real target tables, the real model. Numbers hard-coded in the code are a serious error because they break the TRACEABILITY principle.

3. EVERY NUMBER NEEDS A COMPARISON CONTEXT. "87 tickets" says nothing. "87 tickets, up 12% on last month" or "87 tickets against a target of 100" does. The principle applies to the UI [SE], to the dashboard [DA] and to model-metric reports [AI].

PART A — SE track: React/Vue front end calling a real API

A1. Component structure — split by the screens drawn in the wireframe

The structure below is the minimum. What matters is not the folder names but three separations: the API-calling layer is separate from display components · screen components are separate from reusable components · state is separate from rendering.

Front-end folder structure — React example (Vue is similar)

```
frontend/
├── src/
│   ├── api/                    # API LAYER - every fetch lives here
│   │   ├── client.js           # shared config: base URL, token, error handling
│   │   └── tickets.api.js      # API calls for the ticket business domain
│   ├── hooks/                  # state and data-fetching logic
│   │   └── useTickets.js
│   ├── pages/                  # ONE file per ONE wireframe screen
│   │   ├── TicketListPage.jsx
│   │   ├── TicketDetailPage.jsx
│   │   └── ReceiveDevicePage.jsx
│   ├── components/             # components REUSED across screens
│   │   ├── DataState.jsx       # handles 3 states: loading / empty / error
│   │   ├── Badge.jsx
│   │   └── Pagination.jsx
│   ├── App.jsx
│   └── main.jsx
├── .env.example                # VITE_API_BASE_URL=http://localhost:8000
├── package.json
├── package-lock.json           # MUST be committed
└── README.md
```

Rule: do not scatter fetch calls across components

Every call to the server goes through src/api/. A component never contains the words "fetch" or "axios".

Why: when the base URL changes, when a token is added to the header, when 401 handling is added — there is one place to change, not fifteen.

30-second test: run grep -rn "fetch(\|axios" src/pages src/components — the result must be empty. If not, move that call into src/api/ before Session 10.

A2. The API layer — complete sample code

src/api/client.js — the only place that knows about HTTP

```
const BASE = import.meta.env.VITE_API_BASE_URL;   // NEVER hard-code the URL in code

export class ApiError extends Error {
  constructor(status, code, message) {
    super(message);
    this.status = status;    // HTTP status code
    this.code = code;        // business error code returned by the back end
  }
}

export async function request(path, options = {}) {
  const token = localStorage.getItem('token');
  const res = await fetch(`${BASE}${path}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...options.headers,
    },
  });
```

```
  if (res.status === 204) return null;

  let body = null;
  try { body = await res.json(); } catch { /* response is not JSON */ }

  if (!res.ok) {
    // Translate the server error into an application error, WITH A MESSAGE FOR THE USER
    throw new ApiError(res.status, body?.error, body?.message
      || 'Không kết nối được tới hệ thống. Vui lòng thử lại.');
  }
  return body?.data ?? body;
}
```

src/api/tickets.api.js — one function per business operation; the name says what it does

```
import { request } from './client';

export const ticketsApi = {
  danhSach: ({ trang = 1, trangThai, tuKhoa } = {}) => {
    const q = new URLSearchParams({ trang: String(trang) });
    if (trangThai) q.set('trang_thai', trangThai);
    if (tuKhoa)    q.set('tu_khoa', tuKhoa);
    return request(`/api/tickets?${q}`);
  },
  chiTiet:      id       => request(`/api/tickets/${id}`),
  tiepNhan:     payload  => request('/api/tickets',
                            { method: 'POST', body: JSON.stringify(payload) }),
  phanCong:     (id, ktv) => request(`/api/tickets/${id}/technician`,
                            { method: 'PATCH', body: JSON.stringify({ technicianId: ktv }) }),
};
```

A3. Three states — a reusable component

This is the most worthwhile component in the whole front end: write it once and it removes the three most common mistakes.

src/hooks/useTickets.js and src/components/DataState.jsx

```
// ---------------- hooks/useTickets.js ----------------
import { useEffect, useState } from 'react';
import { ticketsApi } from '../api/tickets.api';

export function useTickets(filters) {
  const [state, setState] = useState({ data: null, loading: true, error: null });

  useEffect(() => {
    let huy = false;
    setState({ data: null, loading: true, error: null });
    ticketsApi.danhSach(filters)
      .then(data => { if (!huy) setState({ data, loading: false, error: null }); })
      .catch(err => { if (!huy) setState({ data: null, loading: false, error: err }); });
    return () => { huy = true; };          // avoid updating after the component has unmounted
  }, [JSON.stringify(filters)]);

  return state;
}

// ---------------- components/DataState.jsx ----------------
export function DataState({ loading, error, data, empty, children }) {
  if (loading) return <div className="skeleton">Đang tải dữ liệu…</div>;

  if (error) return (
    <div className="alert alert-error">
      <p>{error.message}</p>
      {/* IF the error is 422 or 409 it is a business error - tell the user what to do */}
      <button onClick={() => window.location.reload()}>Thử lại</button>
    </div>
  );

  const rong = !data || (Array.isArray(data) && data.length === 0);
```

```
  if (rong) return (
    <div className="empty-state">
      <p>{empty?.message || 'Chưa có dữ liệu nào.'}</p>
      {empty?.action}
    </div>
  );

  return children(data);
}

// ---------------- pages/TicketListPage.jsx (usage) ----------------
export default function TicketListPage() {
  const [filters, setFilters] = useState({ trang: 1 });
  const { data, loading, error } = useTickets(filters);

  return (
    <section>
      <h1>Danh sách phiếu bảo hành</h1>
      <FilterBar value={filters} onChange={setFilters} />
      <DataState loading={loading} error={error} data={data}
        empty={{ message: 'Không có phiếu nào khớp bộ lọc hiện tại.',
                 action: <button onClick={() => setFilters({ trang: 1 })}>Xoá bộ lọc</button> }}>
        {rows => <TicketTable rows={rows} />}
      </DataState>
    </section>
  );
}
```

Why the EMPTY state must distinguish two cases

Empty because there is no data yet: "There are no warranty tickets yet. Click Receive device to create the first one." — the user needs an action to get started.

Empty because the filter matched nothing: "No tickets match the current filter." with a clear-filter button — the user needs to know the problem is the filter, not the system.

Both cases look identical on screen if all you write is "No data", and the user will think the system is broken.

Question the lecturer will ask in Session 10: "Filter by a status that has no tickets." Students should be ready for the screen to show the right message, not a blank table.

A4. Storing and using the token safely at prototype level

How the token is stored	Risk	Allowed in Assignment 2?
Hard-coded in the source	The token lives in the repo; anyone who reads the repo can use it. This is the most serious security mistake and is penalised immediately.	NO. Never.
localStorage	Readable if the page has a script-injection hole (XSS). Acceptable at prototype level if the app does not render unsanitised user input.	Yes, with this limitation recorded in the documentation.
HttpOnly cookie set by the server	Safer, because JavaScript cannot read it. Needs server-side configuration and CORS handling.	Yes, and encouraged. Earns credit under the technical-quality criterion.

Three mandatory authentication items in Assignment 2

The API base URL is in an environment variable, not hard-coded. A .env.example documents the variable.

How the token is stored	Risk	Allowed in Assignment 2?

On a 401 response, automatically clear the token and redirect to the login page — never leave the user staring at an error screen.

Add one line to the documentation stating how the token is stored and the limitation of that choice. Recognising the limits of one's own choices is a CLO5 competency.

A5. Self-check before the Session 10 workshop [SE]

- ☐ At least three main screens work and take data from the real API (not hard-coded data).
- ☐ grep for "fetch(" and "axios" in pages and components returns nothing.
- ☐ All three states are visible: stop the back-end server and open the page → an error message with a retry button appears.
- ☐ Filter by a condition with no results → a dedicated empty message appears with a clear-filter button.
- ☐ The API base URL comes from an environment variable; .env.example exists.
- ☐ A 401 clears the token and redirects to the login page.
- ☐ Tested on a wide screen and a narrow screen (about 375 px) — tables do not overflow horizontally.
- ☐ package-lock.json is committed.
- ☐ The README lists the install and run commands for the front end, noting that the back end must be started first.

PART B — DA track: Loading the data warehouse and building a dashboard

B1. From staging to target tables — re-runnable loads using business keys

In Session 7, students loaded cleaned data into staging tables. The next step is loading into the target tables according to the warehouse schema designed in Assignment 1. The most important requirement: a second load must not duplicate data.

sql/010_load_dim_customer.sql — loading a dimension by business key

```
-- Customer business key: the normalised phone number.
-- Surrogate key customer_sk is generated by the warehouse.
INSERT INTO dim_customer (business_key, ho_ten, sdt_chuan, tinh_thanh, ngay_tao, cap_nhat_luc)
SELECT s.khoa_hop_nhat, s.ho_ten, s.sdt_chuan, s.tinh_thanh, s.ngay_tao, NOW()
FROM   staging_customers s
ON CONFLICT (business_key) DO UPDATE
SET    ho_ten      = EXCLUDED.ho_ten,
       sdt_chuan   = EXCLUDED.sdt_chuan,
       tinh_thanh  = EXCLUDED.tinh_thanh,
       cap_nhat_luc = NOW()
-- Update only on a real change, avoiding pointless rewrites on every run
WHERE  dim_customer.ho_ten     IS DISTINCT FROM EXCLUDED.ho_ten
   OR  dim_customer.sdt_chuan  IS DISTINCT FROM EXCLUDED.sdt_chuan
   OR  dim_customer.tinh_thanh IS DISTINCT FROM EXCLUDED.tinh_thanh;
```

sql/020_load_fact_ticket.sql — loading the fact by batch: delete, then reload

```
-- The fact is loaded by DAILY BATCH. Before loading, delete every row of that batch.
-- This keeps the load re-runnable and is far simpler than row-by-row comparison.
BEGIN;

DELETE FROM fact_ticket
WHERE  ngay_tiep_nhan_key BETWEEN :tu_ngay AND :den_ngay;

INSERT INTO fact_ticket (
  ngay_tiep_nhan_key, customer_sk, product_sk, center_sk, category_sk,
  so_phieu, so_gio_xu_ly, tre_han, chi_phi_linh_kien)
SELECT
  TO_CHAR(t.received_at, 'YYYYMMDD')::INT,
  dc.customer_sk, dp.product_sk, dsc.center_sk, dic.category_sk,
  1                                                  AS so_phieu,
  EXTRACT(EPOCH FROM (t.closed_at - t.received_at))/3600 AS so_gio_xu_ly,
  CASE WHEN t.closed_at > t.due_date THEN 1 ELSE 0 END   AS tre_han,
  COALESCE(lk.chi_phi, 0)                                AS chi_phi_linh_kien
FROM staging_tickets t
JOIN dim_customer      dc  ON dc.business_key  = t.khoa_khach_hang
JOIN dim_product       dp  ON dp.business_key  = t.product_code
JOIN dim_service_center dsc ON dsc.business_key = t.center_code
JOIN dim_issue_category dic ON dic.business_key = t.category_name
LEFT JOIN v_chi_phi_linh_kien lk ON lk.ticket_code = t.ticket_code
WHERE t.received_at::DATE BETWEEN :tu_ngay AND :den_ngay;

COMMIT;
```

Four things that must be true of the fact table — check before drawing charts

1. A clear GRAIN, recorded in the documentation. "One fact row = one warranty ticket" is a grain. If the grain cannot be stated in one sentence, the schema is not right yet and every chart built on it is suspect.

2. Measures must be additive. so_phieu and chi_phi_linh_kien can be summed. A "late rate" CANNOT — store tre_han as 0/1 and compute the rate with AVG at query time; do not store precomputed rates.

3. Use surrogate keys, not business keys, in the fact. The fact references dimensions through customer_sk, so when a customer's phone number changes the fact is unaffected.

4. JOINs must not lose rows. Count staging rows and fact rows after loading. If the fact has fewer, some rows were dropped by a JOIN — find them and handle them with an "unknown" row in the dimension; never drop them silently.

B2. Dashboard — two charts answering two concrete questions

Assignment 2 requires at least two charts. What matters is not the count but that each chart answers an analytical question that can be written as a sentence, and carries a comparison context.

Analytical question	Suitable chart type	Required comparison context	Why this type
Which service centres are slower than the others?	Horizontal bars, sorted descending	System-wide average line, or the SLA target	For comparing discrete categories, horizontal bars make labels easier to read than vertical columns.
How has the late rate changed over six months?	Line over time	Same period last year, or a target threshold drawn as a horizontal line	For trends over time a line shows direction of change; bars do not.
Which issue groups account for most of the parts cost?	Horizontal bars with cumulative percentage	Total cost, so the share of each group can be read	Shows at once which few issue groups make up most of the cost.
Which technicians are overloaded?	Table with in-cell bars, or horizontal bars with a threshold	Threshold of 8 open tickets per business rule QT-11	Drawing the business threshold as a line shows immediately who is over it.

src/dashboard/app.py — Streamlit, three states and a comparison context

```
import streamlit as st
import pandas as pd
import plotly.express as px
from src.db import doc_sql

st.set_page_config(page_title='Mekong Mobile – Bảo hành', layout='wide')
st.title('Hiệu quả xử lý bảo hành')

# --- Filters ---
c1, c2 = st.columns(2)
tu_ngay  = c1.date_input('Từ ngày',  pd.Timestamp('2026-01-01'))
den_ngay = c2.date_input('Đến ngày', pd.Timestamp('2026-06-30'))

@st.cache_data(ttl=300)          # 5-minute cache - DOCUMENT this
def lay_du_lieu(tu, den):
    return doc_sql('sql/q_hieu_qua_trung_tam.sql', {'tu_ngay': tu, 'den_ngay': den})

# --- STATE 1: loading ---
with st.spinner('Đang truy vấn kho dữ liệu…'):
    try:
        df = lay_du_lieu(tu_ngay, den_ngay)
    # --- STATE 3: error ---
    except Exception as e:
        st.error('Không đọc được dữ liệu từ kho. Kiểm tra kết nối và thử lại.')
        st.caption(f'Chi tiết kỹ thuật: {type(e).__name__}')   # NEVER print the connection string
        st.stop()

# --- STATE 2: empty ---
```

Analytical question	Suitable chart type	Required comparison context	Why this type

```
if df.empty:
    st.info('Không có phiếu nào trong khoảng thời gian đã chọn. '
            'Thử mở rộng khoảng ngày hoặc kiểm tra pipeline đã nạp dữ liệu chưa.')
    st.stop()

# --- Has data: draw the chart WITH A COMPARISON CONTEXT ---
tb_he_thong = df['gio_xu_ly_tb'].mean()

fig = px.bar(df.sort_values('gio_xu_ly_tb'),
             x='gio_xu_ly_tb', y='ten_trung_tam', orientation='h',
             labels={'gio_xu_ly_tb': 'Giờ xử lý trung bình',
                     'ten_trung_tam': ''})
# Reference line: this is what turns a chart into a conclusion
fig.add_vline(x=tb_he_thong, line_dash='dash',
              annotation_text=f'Trung bình hệ thống {tb_he_thong:.1f} giờ')
st.plotly_chart(fig, use_container_width=True)
st.caption(f'Nguồn: fact_ticket · {len(df)} trung tâm · '
           f'số liệu tính tới {df["cap_nhat_luc"].max():%d/%m/%Y %H:%M}')
```

Three things every chart in Assignment 2 must have

1. Axis labels with units. "Average handling hours", not "gio_xu_ly_tb". A database column name is not a label for readers.

2. A reference line or marker. An average, a target, the previous period, or a business threshold. Without a reference the viewer cannot conclude anything.

3. A line stating the source and the data timestamp. "Source: fact_ticket · data as of 30/06/2026 14:20". This is the TRACEABILITY principle applied to a dashboard, and it is what separates a usable dashboard from a pretty picture.

B3. Running the pipeline on a schedule — within the scope of this course

This course does not require a full orchestrator (Airflow, Dagster) — that belongs to Specialized Graduation Topic 2. At Assignment 2 level, running on a schedule means: the pipeline runs with a single command that takes date parameters, and how to schedule that command is documented.

Accepted scope for Assignment 2

```
# 1. The pipeline takes date parameters; the default is yesterday
python -m src.pipeline --tu-ngay 2026-06-01 --den-ngay 2026-06-30
python -m src.pipeline                # default: yesterday

# 2. Log every run to a file, WITH the batch id
#    reports/run_20260701_0200.log

# 3. Document scheduling in the Deployment Guide (actual installation not required)
#    Linux/macOS - crontab:
#      0 2 * * *  cd /opt/du-an && ./venv/bin/python -m src.pipeline >> logs/cron.log 2>&1
#    Windows - Task Scheduler: create a daily task at 02:00 with the equivalent command

# 4. When the previous run failed: the pipeline MUST be re-runnable;
#    backfill by calling it again with exactly that date range.
python -m src.pipeline --tu-ngay 2026-06-30 --den-ngay 2026-06-30   # backfill
```

B4. Self-check before the Session 10 workshop [DA]

- ☐ The target tables contain data; row counts are compared with staging and any difference is explained.
- ☐ A second load does not duplicate data (run and re-counted).

- ☐ The grain of the fact table can be stated in one sentence and is recorded in the Data dictionary.
- ☐ Every measure in the fact is additive; no precomputed rates or percentages are stored.
- ☐ There are at least two charts, each tied to an analytical question written as a sentence.
- ☐ Each chart has axis labels with units, one reference marker, and a source line with a timestamp.
- ☐ The dashboard handles all three states: loading, empty (filter returns no rows), connection error.
- ☐ Error messages on the dashboard do NOT print the connection string or internal table names.
- ☐ The pipeline accepts date parameters and logs every run into reports/.
- ☐ The Deployment Guide has a section on scheduling the pipeline.

PART C — AI track: Training, comparing and evaluating models

C1. The most important rule: the test set is used ONCE

Three sets, three different roles — never mix them

Training set (70%): used by the model to learn its parameters. Can be used any number of times.

Validation set (15%): used to CHOOSE — which model, which hyperparameters, which cleaning approach. Can be used many times.

Test set (15%): used EXACTLY ONCE, at the very end, to report results. Every time test results are looked at and the model is then revised, the test set loses value because it has influenced the model choice.

Why this is a matter of honesty, not just convention: if ten models are tried on the test set and the best one is reported, the reported figure is the best of ten attempts, not the model's true ability on new data. Reporting it as true ability misrepresents the results.

C2. Compare at least two models — on the validation set

src/models/train.py — compare on validation, measure once on test

```
import json, joblib
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import ComplementNB
from sklearn.dummy import DummyClassifier
from sklearn.metrics import f1_score, classification_report, confusion_matrix

def huan_luyen_va_chon(X_tr, y_tr, X_val, y_val, hat_giong=42):
    ung_vien = {
        'dummy_da_so':  DummyClassifier(strategy='most_frequent'),
        'logistic':     LogisticRegression(max_iter=1000, class_weight='balanced',
                                           random_state=hat_giong),
        'linear_svc':   LinearSVC(class_weight='balanced', random_state=hat_giong),
        'complement_nb': ComplementNB(),
    }
    ket_qua = {}
    for ten, mo_hinh in ung_vien.items():
        mo_hinh.fit(X_tr, y_tr)
        # CHOOSE on VALIDATION - never on test
        f1 = f1_score(y_val, mo_hinh.predict(X_val), average='macro', zero_division=0)
        ket_qua[ten] = {'f1_macro_val': round(float(f1), 4)}
        print(f'{ten:16s} F1 macro (val) = {f1:.4f}')

    ten_tot_nhat = max(
        (t for t in ket_qua if t != 'dummy_da_so'),
        key=lambda t: ket_qua[t]['f1_macro_val'])
    return ung_vien[ten_tot_nhat], ten_tot_nhat, ket_qua

def do_lan_cuoi(mo_hinh, X_te, y_te, nhan_labels):
    """Call EXACTLY ONCE, after the model has been chosen."""
    y_pred = mo_hinh.predict(X_te)
    return {
        'f1_macro_test': round(float(f1_score(y_te, y_pred, average='macro',
                                              zero_division=0)), 4),
        'f1_theo_nhan': {
            nhan: round(float(f), 4) for nhan, f in zip(
                nhan_labels,
                f1_score(y_te, y_pred, average=None, labels=nhan_labels,
                         zero_division=0))},
        'confusion_matrix': confusion_matrix(y_te, y_pred,
                                             labels=nhan_labels).tolist(),
        'nhan_labels': list(nhan_labels),
        'bao_cao_chi_tiet': classification_report(y_te, y_pred, zero_division=0,
```

```
                                        output_dict=True),
    }
```

C3. Choosing an evaluation metric — and being able to justify it

Metric	What it measures	When to use it	Pitfall
Accuracy	Share of correct predictions out of all predictions	Labels are balanced and all errors are equally bad	With 90/10 skewed data, always guessing the majority label already gives 90% — a high number for a useless model.
F1 macro	Unweighted mean of the F1 of every label	Labels are imbalanced and every label matters equally	Labels with few samples strongly affect the metric, so it fluctuates between data splits.
F1 weighted	Mean F1 weighted by sample count	You want one number reflecting the overall user experience	Hides a model that completely ignores rare labels — closer to accuracy than to F1 macro.
Precision for one label	Of the samples predicted as that label, how many are correct	When false alarms are costly (sending a technician to the wrong place)	Can be very high if the model predicts the label only when extremely sure, missing most real cases.
Recall for one label	Of the samples that truly have that label, how many are found	When misses are costly (missing water damage leads to severe failure)	Can be very high if the model assigns that label to almost every sample.
RMSE / MAE	Average error for regression problems	Predicting continuous values (handling time, cost)	RMSE is dominated by outliers; report MAE alongside it.

Sample answers to "why did you choose this metric"

Weak answer: "F1 macro is used because it is common for classification problems."

Strong answer: "The data is skewed: label KHÁC (other) makes up 31% while NƯỚC_VÀO (water damage) is only 9%. With accuracy, a model that always guesses the majority label already scores 31%, so that number says nothing. F1 macro was chosen because it averages all six labels equally, so if the model neglects NƯỚC_VÀO the metric drops visibly. From a business point of view, missing a water-damage case is the most costly error, so the report also gives the recall of that label separately."

The difference: the strong answer ties the metric to (1) real characteristics of the data, with numbers; and (2) business consequences. That is traceability.

This is exactly the type of question in item 2c of the final exam and in the standard three questions of Session 10.

C4. Reading the confusion matrix — turning a table of numbers into conclusions

The confusion matrix is the most powerful diagnostic tool students have, but only when it is actually read rather than pasted into the report. How to read it: each row is a true label, each column a predicted label. Off-diagonal cells are mistakes.

Sample confusion matrix and how to read three conclusions from it

```
Predicted label →  MÀN_HÌNH  PIN   SẠC   PHẦN_MỀM  NƯỚC_VÀO  KHÁC
```

```
True label
  MÀN_HÌNH          142     3     2       4         6       11
  PIN                 4   118    31       3         1       14
  SẠC                 2    28    96       2         0       12
  PHẦN_MỀM            5     4     3      88         1       29
  NƯỚC_VÀO            9     2     1       2        41       16
  KHÁC               14    12    10      24         7      113

THREE CONCLUSIONS FROM THIS TABLE:

1. The most confused pair is PIN ⇔ SẠC (battery ⇔ charging) (31 + 28 = 59).
   Business meaning: battery and charging fault descriptions share many words
   ("won't power on", "won't charge"). Remedy: add distinguishing features,
   or merge the two labels if the business accepts it - and
   IF MERGED, justify it to the business users; do not decide alone.

2. Label NƯỚC_VÀO (water damage) has the lowest recall: 41/(9+2+1+2+41+16) = 57%.
   Business meaning: this is the MOST COSTLY label to miss (a wet device
   left too long is damaged more). The 43% missed cases are a real business risk
   and must be stated in the report and in the model card.

3. Label KHÁC (other) absorbs many wrong predictions from all labels (column KHÁC: 11+14+12+29+16).
   Meaning: KHÁC is acting as a garbage label. Remedy: revisit the label definition
   with business users, or exclude KHÁC from the main metric and report it separately.
```

Three sentences to write in the Assignment 2 report after reading the confusion matrix

Which pair of labels is confused most often, and what does that mean for the business?

Which label has the lowest recall, and what is the real-world consequence of missing it?

Is any label acting as a "garbage bin" that absorbs every uncertain prediction?

These three sentences are the most valuable part of the report. A table of numbers pasted into a report without these three sentences earns almost nothing under the technical-quality criterion.

C5. Saving the model — but not committing the model file

Saving and reloading a model correctly

```
import joblib, json, pathlib

# --- Save: SAVE BOTH the vectorizer AND the model, together, same version ---
pathlib.Path('artifacts').mkdir(exist_ok=True)
joblib.dump({'vectorizer': vec, 'model': mo_hinh_tot_nhat,
             'nhan_labels': list(nhan_labels),
             'hat_giong': cfg['hat_giong'],
             'phien_ban_du_lieu': cfg['phien_ban_du_lieu']},
            'artifacts/model_v1.joblib')

# --- .gitignore MUST contain: ---
# artifacts/*.joblib
# artifacts/*.pkl

# --- Instead, commit reports/metrics.json (a small, readable text file) ---
# and put the command that regenerates the model in the README:
#   python -m src.models.train --config configs/train.yaml
```

Why the model file is not committed to the repo

Model files are often tens of megabytes and binary — the repo bloats and every change stores a full new copy.

Binary files cannot be diffed between versions, so they do not belong in source version control.

Instead: commit the script that produces the model + the random seed + exact library versions. Anyone who reruns the script gets exactly that model. This is the REPRODUCIBILITY principle.

If the model file must be submitted, submit it through the LMS as a separate attachment, not in the repo.

C6. Self-check before the Session 10 workshop [AI]

- ☐ A majority-class baseline has been trained and its score recorded in metrics.json.
- ☐ At least two real models have been trained and compared.
- ☐ Model SELECTION is based on the validation set, not on the test set.
- ☐ The test set is measured only once, after the model has been chosen.
- ☐ metrics.json contains F1 macro, per-label F1 and the confusion matrix.
- ☐ The three conclusions from the confusion matrix are written (most confused pair, lowest-recall label, garbage label).
- ☐ The choice of metric can be explained in words, tied to the real label distribution.
- ☐ The random seed is recorded and two runs give identical results.
- ☐ The vectorizer and the model are saved in the same artifacts file, and .gitignore blocks it.
- ☐ requirements.txt pins exact versions with ==.

PART D — Common to all three tracks: contracts at boundaries and incremental integration

D1. Write the data contract for the most important boundary of the module

The Session 9 lecture listed three things every contract must state: SHAPE (fields, types, required or not), MEANING (what values and units mean) and BEHAVIOUR ON FAILURE (what is returned when there is no data, how errors are reported). The template below works for all three tracks: put it directly above the boundary conversion function, and write a test that checks exactly what the comment promises.

Contract comment template — SE (JavaScript), DA and AI (Python)

```
// [SE] src/service/ticket.contract.js
// CONTRACT: one ticket returned by the business layer to the presentation layer.
//   hanCamKet   : ISO 8601 string with time zone, always present
//   kyThuatVien : string OR null if not yet assigned
//   tenKhach    : string, phone number masked per QT-15
// ON FAILURE: ticket not found -> throw NotFoundError (controller returns 404).

# [DA] src/transform/schema.py
# CONTRACT: ticket DataFrame after extract, before transform.
#   ticket_code : str, unique, not empty
#   received_at : timezone-aware datetime, Asia/Ho_Chi_Minh
#   so_gio_xu_ly: float, unit HOURS, NaN if the ticket is still open
# ON FAILURE: missing column -> stop the pipeline and log; bad row -> reports/reject.csv

# [AI] src/features/build.py
# CONTRACT: build_features(df) is SHARED by training and inference.
#   Output  : exactly FEATURE_COLUMNS, in that order, no NaN
# ON FAILURE: missing mo_ta_loi -> KeyError; empty description -> empty string, row kept
```

D2. Incremental integration — join one seam at a time

The main flow of Assignment 2 passes through several seams. Joining everything and only then running it (big-bang) produces an error of unknown origin. Joining one seam at a time and running after each join means that, when something breaks, the fault is in the seam just added. Any downstream part that is not ready yet is temporarily replaced by a stub that returns sample data matching the contract.

The seams of the main flow by track

[SE] UI → API client → controller → service → repository → database. Typical stub: a fake API returning the exact ticket shape so the UI can be built first.

[DA] source files → extract → staging → transform → target tables → dashboard. Typical stub: a small hand-loaded fact table so the dashboard can be built while the pipeline is unfinished.

[AI] data → build_features → model → metrics.json (→ inference API if any). Typical stub: a DummyClassifier standing in for the real model so the whole flow can be connected.

Note: a stub is temporary scaffolding. The Assignment 2 submission must NOT still contain stubs in the main flow.

D3. Common self-check before the Session 10 workshop

- ☐ The most important boundary of the module has a contract comment stating shape, meaning and behaviour on failure.

- ☐ There is a conversion or normalisation function at the boundary; renaming a field means changing one place only.
- ☐ Input is validated at the boundary (zod, Joi, pydantic, pandera or assert), not with ifs scattered across the business layer.
- ☐ [SE] The API returns errors in one consistent format with error, message and fields; user errors return 4xx, never 500.
- ☐ There is at least one contract test: field names, date format, null values [SE]; column names [DA]; column order [AI].
- ☐ At least two adjacent seams of the main flow have been joined incrementally and run successfully.

Quick comparison of the three tracks — a self-check reference table

Criterion	SE track	DA track	AI track
Output of Sessions 9–10	≥ 3 screens calling the real API	Target tables loaded + ≥ 2 charts	≥ 2 models compared + metric report
Where the three states appear	DataState component: loading / empty / error	Dashboard: spinner / empty message / error message	Not applicable to a UI, but applies to the report: state clearly which labels lack data
"Re-runnable" check item	Not applicable	A second load does not duplicate rows	Two runs produce identical metrics.json
What the comparison context is	On-screen figures show the previous period or a target	Reference marker on the chart	Baseline model for comparison
Most serious mistake	Hard-coded data in a component instead of calling the API	Charts with no reference marker and no source	Using the test set to choose the model
Question the lecturer often asks in Session 10	"Stop the back end and open the page — what happens?"	"Up to what point in time are these figures?"	"Why this metric rather than accuracy?"

Three things to do this week, before the Session 10 workshop

1. Get the ERROR STATE of your track working. [SE] stop the back end and open the page · [DA] break the connection string and open the dashboard · [AI] run training with a data file that does not exist. All three must produce a clear message, not a long stack trace.

2. Write ONE question that your presentation answers, as a full sentence. If it cannot be written as a sentence, the presentation has no purpose yet and needs rethinking.

3. Add ONE reference marker. An average line, a target, a baseline model. It takes fifteen minutes and it is what turns numbers into a conclusion.